

VOTARA

Decentralized Voting Platform on Solana

24-Hour Execution Plan

Project	Votara - Fast, Secure & Transparent Decentralized Voting
Platform	Solana Blockchain
Team Size	Solo Developer
Timeline	24 Hours (0 → Production Ready)
Hackathon	Solana Frontier 2025
Date	May 09, 2026

1. EXECUTIVE SUMMARY

Votara is a next-generation decentralized voting platform built on Solana, designed for DAOs, communities, and digital governance. Leveraging Solana's high throughput (65,000+ TPS) and sub-second finality, Votara enables real-time, transparent, and secure voting at scale.

Key Innovation: Helius LaserStream Integration

Votara differentiates itself by integrating **Helius LaserStream** (gRPC-based real-time blockchain streaming) for instant vote updates and live proposal tracking. This provides a Netflix-like real-time experience for governance, something traditional voting platforms cannot achieve. This unique dependency will surprise judges with sub-100ms vote confirmation visibility.

Core Features:

- ✓ Token-gated voting with SPL token integration
- ✓ Real-time vote tracking via Helius LaserStream (gRPC)
- ✓ On-chain treasury management with multisig support
- ✓ Quadratic voting support for equitable governance
- ✓ Time-locked proposals with automatic closure
- ✓ Sybil resistance through Phantom wallet integration
- ✓ Sub-second vote finality (Solana's 400ms blocks)
- ✓ Mobile-responsive PWA with offline vote queuing

2. COMPLETE TECH STACK

2.1 Blockchain & Smart Contracts

Component	Technology	Version/Details
Blockchain	Solana	Mainnet-beta / Devnet
Smart Contract Framework	Anchor	v0.30.1
Programming Language	Rust	1.75.0+
RPC Provider	Helius	Developer Plan (\$24.50/mo - 50% discount)
★ UNIQUE	Helius LaserStream	gRPC real-time streaming (exclusive)
WebSocket	Enhanced WebSockets	On-chain event streaming
Token Standard	SPL Token	Metaplex Token Metadata
Wallet Adapter	Solana Wallet Adapter	Multi-wallet support

2.2 Frontend & User Interface

Component	Technology	Version/Details
Framework	React	18.3.1 (with Vite)
Build Tool	Vite	5.4.2 (Fast HMR)
Language	JavaScript/JSX	ES2022+
State Management	React Context API	Built-in hooks
Wallet Integration	@solana/wallet-adapter-react	Latest
UI Components	Tailwind CSS + shadcn/ui	Responsive design
Icons	Lucide React	Modern icon library
Real-time Updates	Helius gRPC Client	LaserStream integration
Notifications	React Hot Toast	User feedback
Charts/Analytics	Recharts	Vote visualization

2.3 Development & Testing Tools

Component	Technology	Purpose
Package Manager	npm/yarn	Dependency management
Linting	ESLint	Code quality
Testing	Anchor Test Suite	Smart contract testing
Local Validator	Solana Test Validator	Local development
CLI Tools	Solana CLI + Anchor CLI	Deployment & management
Browser Extension	Phantom Wallet	Testing & authentication

Version Control	Git + GitHub	Code repository
Deployment	Vercel/Netlify	Frontend hosting

3. 24-HOUR EXECUTION TIMELINE

This aggressive timeline is optimized for a solo developer with existing Solana knowledge. Each phase has clear deliverables and success criteria.

PHASE 1: Setup & Smart Contract Core (Hours 0-4)

Time	Task	Deliverable
00:00-00:30	Environment Setup	- Anchor project initialized - Helius API key configured - Git repository created
00:30-01:30	Core Smart Contract (vote_app program)	- Account structures defined: - Voter account - Proposal account - Treasury account - PDAs configured
01:30-03:00	Voting Logic Implementation	- register_voter instruction - register_proposal instruction - cast_vote instruction - Token-gating logic
03:00-04:00	Treasury & Token Logic	- initialize_treasury instruction - buy_tokens instruction - SPL token integration - Basic tests passing

Success Criteria: Smart contract compiles, deploys to devnet, basic tests pass

PHASE 2: Advanced Features & Testing (Hours 4-8)

Time	Task	Deliverable
04:00-05:00	Advanced Instructions	- close_proposal (time-lock) - pick_winner (random selection) - withdraw_sol instruction
05:00-06:00	Helius Integration Setup	- LaserStream gRPC client - Subscribe to program updates - Real-time vote streaming

06:00-07:30	Comprehensive Testing	• Unit tests (all instructions) • Integration tests • Edge case testing • Security audit checklist
07:30-08:00	Devnet Deployment & IDL	• Deploy to Solana devnet • Generate IDL JSON • Verify on Solana Explorer • Document program ID

Success Criteria: All tests pass, contract deployed, LaserStream receiving events

PHASE 3: Frontend Development (Hours 8-14)

Time	Task	Deliverable
08:00-09:00	React Project Setup	• Vite + React initialized • Wallet adapter configured • Constants file (program ID, RPC) • IDL imported
09:00-10:30	Core Components (Part 1)	• RegisterVoter.jsx • RegisterProposal.jsx • Vote.jsx • VoterInfo.jsx
10:30-12:00	Core Components (Part 2)	• AllProposals.jsx • ProposalInfo.jsx • BuyTokens.jsx • TokenBalance.jsx
12:00-13:00	Treasury Components	• InitializeTreasury.jsx • TreasuryInfo.jsx • WithdrawSol.jsx
13:00-14:00	Admin Components	• CloseProposal.jsx • CloseVoter.jsx • PickWinner.jsx • Component integration

Success Criteria: All components functional, wallet connects, transactions succeed

PHASE 4: UI/UX & Real-time Features (Hours 14-18)

Time	Task	Deliverable
14:00-15:30	Tailwind CSS Styling	• Responsive layout • Solana-themed colors • Mobile optimization • Dark mode support
15:30-17:00	LaserStream Frontend Integration	• gRPC client in React • Real-time vote updates • Live proposal status • WebSocket fallback
17:00-18:00	Charts & Analytics	• Recharts integration • Vote distribution graphs • Proposal timeline viz • Treasury balance display

Success Criteria: Professional UI, real-time updates working, charts displaying data

PHASE 5: Testing, Optimization & Documentation (Hours 18-22)

Time	Task	Deliverable
18:00-19:00	End-to-End Testing	• Full user flow testing • Multi-wallet testing • Error handling • Transaction retries
19:00-20:00	Performance Optimization	• Bundle size optimization • Code splitting • Image optimization • Caching strategies
20:00-21:00	Security Hardening	• Input validation • Transaction simulation • Rate limiting • Security best practices
21:00-22:00	Documentation	• README.md (setup guide) • Architecture diagram • API documentation • User guide with screenshots

Success Criteria: No critical bugs, optimized performance, comprehensive docs

PHASE 6: Deployment & Submission (Hours 22-24)

Time	Task	Deliverable
22:00-22:30	Production Build	• Vite production build • Environment variables set • Asset optimization
22:30-23:00	Deployment	• Deploy to Vercel/Netlify • Custom domain setup • SSL certificate
23:00-23:30	Demo Video Creation	• Screen recording • Feature walkthrough • LaserStream demo • 3-minute cut
23:30-24:00	Final Submission	• GitHub repository clean • Submission form complete • Social media announcement • Backup deployment

Success Criteria: Live production deployment, demo video published, submission complete

4. UNIQUE DIFFERENTIATOR: HELIUS LASERSTREAM

Why Helius LaserStream Will Impress Judges:

- 1. Real-time Vote Streaming (gRPC):** Unlike traditional polling or WebSocket approaches, LaserStream uses gRPC to stream transaction data at sub-100ms latency. Judges will see votes appear instantly without page refresh.
- 2. Exclusive Helius Feature:** LaserStream is available ONLY on Helius RPC, making it a unique technical choice that demonstrates deep knowledge of Solana's infrastructure ecosystem.
- 3. Production-grade Infrastructure:** Helius provides dedicated nodes with staked connections, meaning our votes get priority inclusion in blocks—faster than competitors using generic RPCs.
- 4. Enhanced Methods:** We'll use Helius's exclusive `getTransactionsForAddress` method to display complete voting history for any wallet, creating a transparent audit trail.
- 5. Cost Efficiency:** At \$24.50/month (50% Frontier discount), we get enterprise-grade infrastructure that would cost \$500+/month on AWS/GCP. This demonstrates smart resource allocation for a hackathon project.

Technical Implementation:

The frontend will establish a persistent gRPC connection to Helius LaserStream, subscribing to our program's account updates. When any vote transaction is processed, the update streams to all connected clients in real-time. This creates a collaborative, live governance experience similar to Google Docs for voting.

5. RISK MITIGATION & CONTINGENCY PLANS

Risk	Probability	Impact	Mitigation Strategy
LaserStream integration complexity	Medium	High	Build basic WebSocket fallback; LaserStream is optional enhancement
Smart contract bugs	Medium	Critical	Extensive testing phase; use Anchor's safety features; security checklist
Devnet instability	Low	Medium	Have localnet validator ready; quick switch capability
Time overrun	Medium	High	Prioritized feature list; MVP by Hour 20, polish in final 4 hours
Frontend styling delays	Low	Low	Use shadcn/ui pre-built components; minimal custom CSS
Wallet connection issues	Low	Medium	Test with Phantom + Backpack; fallback to basic wallet
Deployment failures	Low	Medium	Prepare Vercel + Netlify configs; test deploy at Hour 20

6. SUCCESS METRICS & DELIVERABLES

Minimum Viable Product (MVP) Requirements:

- ✓ Smart contract deployed to Solana devnet
- ✓ All 14 components functional and tested
- ✓ Wallet connection working (Phantom)
- ✓ Vote casting with real-time updates
- ✓ Token-gated proposal creation
- ✓ Treasury initialized and functional
- ✓ Responsive UI (desktop + mobile)
- ✓ Production deployment on Vercel/Netlify
- ✓ README with setup instructions
- ✓ 3-minute demo video

Stretch Goals (If Time Permits):

- Quadratic voting implementation
- Multi-signature proposal approval
- NFT-gated voting (Metaplex integration)
- Proposal delegation feature
- Advanced analytics dashboard
- PWA with offline support

7. RESOURCE REQUIREMENTS & COSTS

Resource	Provider	Cost	Notes
RPC Infrastructure	Helius Developer Plan	\$24.50/mo	50% Frontier discount (normally \$49)
Frontend Hosting	Vercel Free Tier	\$0	Unlimited bandwidth for hobby projects
Domain Name	Namecheap/Cloudflare	\$12/year	Optional; can use .vercel.app subdomain
Smart Contract Deploy	Solana Devnet	\$0	Free devnet SOL from faucet
Development Tools	VS Code + Extensions	\$0	Free and open source
Testing SOL	Devnet Faucet	\$0	Unlimited devnet SOL
TOTAL COST		~\$25-37	One-time setup + 1 month operation

8. CRITICAL PATH ITEMS

These tasks **MUST** be completed in sequence and cannot be parallelized or skipped:

1. **Anchor program deployment** → Generates program ID needed for frontend
2. **IDL generation** → Frontend cannot interact with contract without IDL
3. **Helius API key activation** → Required for LaserStream and RPC calls
4. **Core voting logic tested** → Cannot build UI without working backend
5. **Wallet adapter configured** → All components depend on wallet connection
6. **Production build test** → Catches environment issues before final deployment

9. IMPLEMENTATION BEST PRACTICES

Smart Contract Best Practices:

- Use Anchor's constraint macros (`#[account(...)]`) for automatic validation
- Implement proper error handling with custom error codes
- Add `require!` statements for all business logic checks
- Use `Clock::get()?.unix_timestamp` for time-based logic (proposal expiry)
- Implement proper PDA derivation with seeds for deterministic addresses
- Add comprehensive logging with `msg!()` for debugging
- Test all instructions with both success and failure cases

Frontend Best Practices:

- Implement loading states for all async operations
- Add error boundaries to catch React component errors
- Use `React.memo()` for expensive components to prevent re-renders
- Implement transaction retry logic with exponential backoff
- Display clear user feedback with toast notifications
- Add transaction confirmation UI before submission
- Implement proper wallet connection error handling
- Cache RPC responses where appropriate to reduce calls

10. PRE-DEPLOYMENT CHECKLIST

- All smart contract tests passing (100% coverage)
- Devnet deployment verified on Solana Explorer
- Frontend connects to devnet (not localhost)
- Environment variables configured (`.env.production`)
- Helius RPC endpoint responding (test API key)
- All wallet interactions tested (connect, sign, disconnect)
- Error handling tested (rejected transactions, network failures)
- Mobile responsiveness verified (Chrome DevTools)
- Production build completes without warnings
- README.md updated with live demo link
- Demo video recorded and uploaded

- GitHub repository public and well-organized
- Social media assets prepared (screenshots, description)
- Submission form filled out completely

11. EMERGENCY RESOURCES & QUICK REFERENCES

Resource	URL/Command	Purpose
Solana Devnet Faucet	https://faucet.solana.com	Get free devnet SOL
Helius Dashboard	https://dashboard.helius.dev	API keys, usage stats
Solana Explorer	https://explorer.solana.com?cluster=devnet	Verify transactions
Anchor Docs	https://www.anchor-lang.com	Framework reference
Solana Cookbook	https://solanacookbook.com	Code snippets
Deploy Command	<code>anchor deploy --provider.cluster devnet</code>	Deploy program
Test Command	<code>anchor test</code>	Run all tests
Build Frontend	<code>npm run build</code>	Production build
Local Dev	<code>npm run dev</code>	Start dev server
Wallet Connect Issue	Refresh page + reconnect wallet	Common fix
RPC Rate Limit	Switch to Helius or add delay	Avoid 429 errors

12. FINAL THOUGHTS & STRATEGY

Focus on the Demo: Judges spend ~5 minutes per project. Our LaserStream real-time voting demo must be immediately visible and impressive. Open with this feature.

Embrace Solana's Strengths: Highlight the 400ms block times, sub-cent transaction costs, and parallel processing. These make Votara fundamentally superior to Ethereum-based governance platforms.

Story Matters: Frame Votara as solving real DAO governance pain points: slow Snapshot votes, opaque tallying, expensive gas fees. We're not just building tech—we're democratizing governance.

Solo Developer Advantage: No coordination overhead, no merge conflicts, no communication delays. Every hour is pure execution. This timeline is aggressive but achievable for a focused solo dev.

Fail Fast Philosophy: If LaserStream integration takes >3 hours, fall back to basic WebSockets. MVP over perfection. Ship a working product, iterate later.

Good luck with the build! Remember: 'Perfection is the enemy of shipped.' Focus on core functionality first, polish second. You got this! ■

APPENDIX A: COMPLETE FILE STRUCTURE

```
vote_app/ ■■■ client/ # Frontend React application ■■■ public/ ■ ■■■ vite.svg ■■■ src/ ■ ■■■
assets/ # Images, logos ■ ■■■ components/ # React components (14 total) ■ ■ ■■■ AllProposals.jsx # Display
all active proposals ■ ■ ■■■ BuyTokens.jsx # SPL token purchase ■ ■ ■■■ CloseProposal.jsx # Admin: close
voting ■ ■ ■■■ CloseVoter.jsx # Admin: close voter account ■ ■ ■■■ InitializeTreasury.jsx ■ ■ ■■■
PickWinner.jsx # Random winner selection ■ ■ ■■■ ProposalInfo.jsx # Single proposal details ■ ■ ■■■
RegisterProposal.jsx # Create new proposal ■ ■ ■■■ RegisterVoter.jsx # Voter registration ■ ■ ■■■
TokenBalance.jsx # Display user tokens ■ ■ ■■■ TreasuryInfo.jsx # Treasury stats ■ ■ ■■■ Vote.jsx # Cast
vote on proposal ■ ■ ■■■ VoterInfo.jsx # Voter account details ■ ■ ■■■ WithdrawSol.jsx # Treasury
withdrawal ■ ■ ■■■ constants/ ■ ■ ■■■ constants.js # Program ID, RPC endpoints ■ ■ ■■■ idl/ ■ ■ ■■■
idl.json # Generated from Anchor ■ ■ ■■■ App.css # Global styles ■ ■ ■■■ App.jsx # Main app component ■ ■ ■■■
index.css # Tailwind imports ■ ■ ■■■ main.jsx # React entry point ■ ■ ■■■ polyfills.js # Browser compatibility
■■■ .gitignore ■■■ package.json # Frontend dependencies ■■■ vite.config.js # Vite configuration ■■■
index.html ■■■ programs/vote_app/ # Anchor smart contract ■■■ src/ ■ ■ ■■■ lib.rs # Main program logic ■
■■■ state.rs # Account structures ■ ■ ■■■ instructions/ # Instruction handlers ■ ■ ■■■ register_voter.rs
■■ ■■■ register_proposal.rs ■ ■ ■■■ cast_vote.rs ■ ■ ■■■ close_proposal.rs ■ ■ ■■■
initialize_treasury.rs ■ ■ ■■■ ... ■ ■ ■■■ errors.rs # Custom error codes ■ ■■■ Cargo.toml ■■■ tests/ #
Integration tests ■■■ vote_app.ts ■■■ migrations/ ■■■ Anchor.toml # Anchor configuration ■■■ Cargo.toml #
Workspace config ■■■ package.json # Project scripts
```

APPENDIX B: COMPLETE DEPENDENCY LIST

Backend (Rust/Anchor):

```
[dependencies] anchor-lang = "0.30.1" anchor-spl = "0.30.1" solana-program = "1.18"
spl-token = "4.0"
```

Frontend (package.json):

```
{ "dependencies": { "@solana/web3.js": "^1.95.8", "@solana/wallet-adapter-react": "^0.15.35",
"@solana/wallet-adapter-react-ui": "^0.9.35", "@solana/wallet-adapter-wallets": "^0.19.32",
"@project-serum/anchor": "^0.30.1", "@helius-labs/helius-sdk": "^1.3.4", "react": "^18.3.1",
"react-dom": "^18.3.1", "react-hot-toast": "^2.4.1", "recharts": "^2.13.3", "lucide-react":
"^0.462.0", "buffer": "^6.0.3" }, "devDependencies": { "@vitejs/plugin-react": "^4.3.4", "vite":
"^5.4.2", "tailwindcss": "^3.4.17", "autoprefixer": "^10.4.20", "postcss": "^8.4.49", "eslint":
"^9.17.0" } }
```